

MAE 3780: Mechatronics — Robot Project Final Report

Group 37

Poon Jeerapat (pj259) | Angus Chang (sc3295) | Isaac Ng (in63)
Spring 2026

13 May 2026

1. Robot Design and Strategy Overview

Our robot was designed for the Cube Craze competition, where two robots compete in the span of one minute to collect as many 1-inch wooden cubes as possible that were placed along the midline. 20 cubes are scattered in the middle of the field divided into blue and yellow halves, with black border marking the perimeter. Our design is based on three goals: maximize the sweep width, reliably detecting the midline, and maintaining continuous motion for 60 seconds.

Mechanically, the robot utilizes the standard “Boe-Bot” chassis with two driven wheels and a rear ball caster for stability. The major addition to the design is the U-shaped 3D-printed block guard that maximizes the robot's range in collecting and storing the cubes and also the servo mechanism, spinning the pipe cleaner attached to the metal rod. The pipe cleaner bristles spin to actively sweep cubes inward and prevent them from escaping. The brush and guard features are both essential for retaining the cubes during aggressive lateral sweeps. CAD details and additional photos are provided in Appendix C.

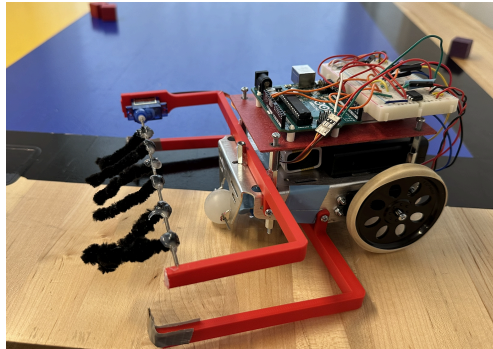


Figure 1: Competition-ready robot showing U-shaped plow with servo-driven pipe cleaner brush mechanism.

Electrically, the robot is controlled using an Arduino Uno. Two DC motors are driven through L9110H H-bridge ICs, controlled via direct PORTB register manipulation on digital pins 8-11 for fast, simultaneous switching. The color sensor is mounted on the chassis under the robot, and is also front oriented to ensure early reading for fast response and early action.

For the code inputs, we implemented mainly two phase states for this robot. In Phase 1, the robots would drive straight forward until it encountered the midline, or when it detected the difference in colors. This works when transitioning from yellow to blue or vice versa. After the robot crosses the midline, it executes a 90 degrees turn(450ms) to orient along it. In Phase 2, the robot drives laterally, pushing cubes as it goes. This action continues upon detecting the black border, it reverses for 400 ms, performs a 180 degrees turn(900ms), and sweeps back. This repeats until the match ends. Color thresholds were calibrated empirically: Black < 400, Yellow 400 – 660, and Blue > 660. Commented code is in Appendix E.

2. Design Process Reflection

Our design has improved significantly through iterative prototyping, particularly on the software aspect and block-storing guard. Our initial brainstorming meetings focused on the main block storing mechanism as well as our block retaining mechanism. Once we finalized the top and bottom guard components, we shifted our focus to integrating the electric components and writing the code.

We discussed and explored alternative manufacturing methods for building the top and bottom guards. In the end, we settled with the idea of using 3D printed parts as they are fully customizable to ensure our design obeys the rules and it is also durable and light. An initial concern was the pricing of the 3D printed components, but we mitigated these concerns by redesigning our components to be lighter and using low infill % to decrease the mass of the printed components. These design changes are visible in the differences between figure 4: *“Final U-Shaped Block Guard” CAD with sliced mass estimate used in BOM* and figure 5: *“Prototype Block Guard” CAD with sliced mass estimate used in BOM*. We used around 7% infill for each of our components because it would be rigid enough to support blocks and light enough to cost less. An initial CAD of the guard and sweeper were used to complete milestone 1 to demonstrate our idea, but the CAD and physical components were not finalized until Milestone 4 when we needed to demonstrate their ability to collect blocks.

The most time-consuming challenge was the color sensor calibration. We used `pulseIn()` to measure the TCS3200 output period directly. This shows unreliable data: yellow surfaces produce pulses too short for accurate measurement, while black surfaces produce pulses too long that trigger timeouts. After several debugging sessions, we came to the conclusion that we would fix the counting rising edges within a fixed 50ms window. This gave stable, well-separated readings.

However, the most persistent issue we encountered is that the reading from the color sensor when we plugged into the computer is not the same as when the robot is independent and the arduino is powered using a 9V battery. This poses the problem as our code was not working properly and very unreliable. Sometimes it was able to detect the color differently and not in some runs. We then took a few steps back and checked every part in our robot. We then found out the root cause of the problem is the battery powering the Arduino UNO. After measuring the battery with a multimeter, it shows only 6.6 V. We switched the battery to a new one. The LED light of the color sensor lit up even brighter and it read the appropriate pulses.

In addition, our strategy also pivoted. We initially coded a U-shaped path (drive to border, turn 90 degree, cross to opposite color, turn 90 degree, drive forward), but our robot was relatively slow so by the time it is sweeping the midline, the other team might already collect all cubes. Therefore, we came up with a final strategy of going straight to the midline, grab most of the proportion and keep collecting while retaining the already-collected cubes.

3. Competition Analysis

On the competition day, our robot won 5 of 7 rounds, tied 1, and lost 1. Given the sensor calibration difficulties during the development, we were satisfied with this result.

The midline sweep strategy worked well and turned out to be very reliable. Also, we found it fascinating that our robot out-powered most of the teams' robots. This could be that our motor is not modified for more speed and it could provide more torque when compared to the modified ones.

Our primary strength was sweep coverage. The U-shaped plow covered roughly twice the chassis width, and the pipe cleaner brush actively retained the cubes. We collected an average of about 5 cubes per match, and peaked at around 7 blocks.

Our loss came against a fast opponent that reached the cubes before us and pushed the cube almost out of the border. Speed is our weakness. The 1 tie came from the first round where our plow tangled with the opponent's mechanism, immobilizing both robots for the remaining time.

Another improvement that could be made upon the robot is that we thought the brush is adequate to prevent the cubes from escaping during the 180 degree turn. However, the reversal motion scattered the cube and pushed the cube to the front edge of the plow. We lost a lot of collected cubes during the match due to the height difference between the field platform and the ground. In future designs we could look into building an effective cube storage in the robot, instead of letting them scattered and become disorganized.

4. Conclusions

This project reinforced the importance of robust sensor calibration and systematic hardware debugging in embedded systems. Transitioning from raw measurements to a fixed 50ms pulse counting window was a pivotal breakthrough for our TCS3200 color sensor, allowing us to reliably detect the midline and boundaries. Furthermore, diagnosing erratic system behavior back to a depleted 6.6 V battery highlighted the critical dependency between stable power delivery and logic level sensor accuracy.

Mechanically, our iterative design approach proved highly effective. We managed to construct a wide, U-shaped plow that maximized our sweep coverage while remaining well within budget constraints. Coupled with the active servo driven pipe cleaner brush and our strategy of direct midline interception, the robot achieved a strong 5-1-1 competition record, peaking at 7 collected cubes in a single match.

While our motors provided excellent torque for pushing through areas, our lack of top speed cost us against faster opponents who reached the centerline first. Additionally, the open plow design, though effective for forward sweeping, struggled with cube retention during our 180 degree reversal sequences and when navigating the field's height difference. Future iterations would benefit from optimizing the drivetrain for high RPM and designing a fully enclosed storage compartment to secure the cubes during complex directional changes. For future students, we recommend verifying your battery voltage with a multimeter whenever erratic sensor behavior occurs, as a depleted battery can easily disguise itself as a complex software bug. Additionally, when calibrating the TC3200 color sensor, moving away from raw measurements to a fixed pulse count will save hours of debugging and yield significantly more stable results.

5. Appendix A: Bill of Materials

Component	Quantity	Source	Price per Unit	Total Cost
18-8 Stainless Steel Threaded Rod	1	McMaster	5.08	5.08
Pipe Cleaners	10	Scavenged	0.25	2.5
Final U-Shaped Block Guard	1	RPL	7.684	7.684
U-Shaped Servo Holder	1	RPL	5.9	5.9
Prototype Block Guard	1	RPL	13.56	13.56
				34.724

Table 1: Bill of Materials including all purchased, manufactured, and scavenged components. All RPL component costs were calculated using the course's ABS price = $(\text{Mass} \times 0.4) + 1$

6. Appendix B

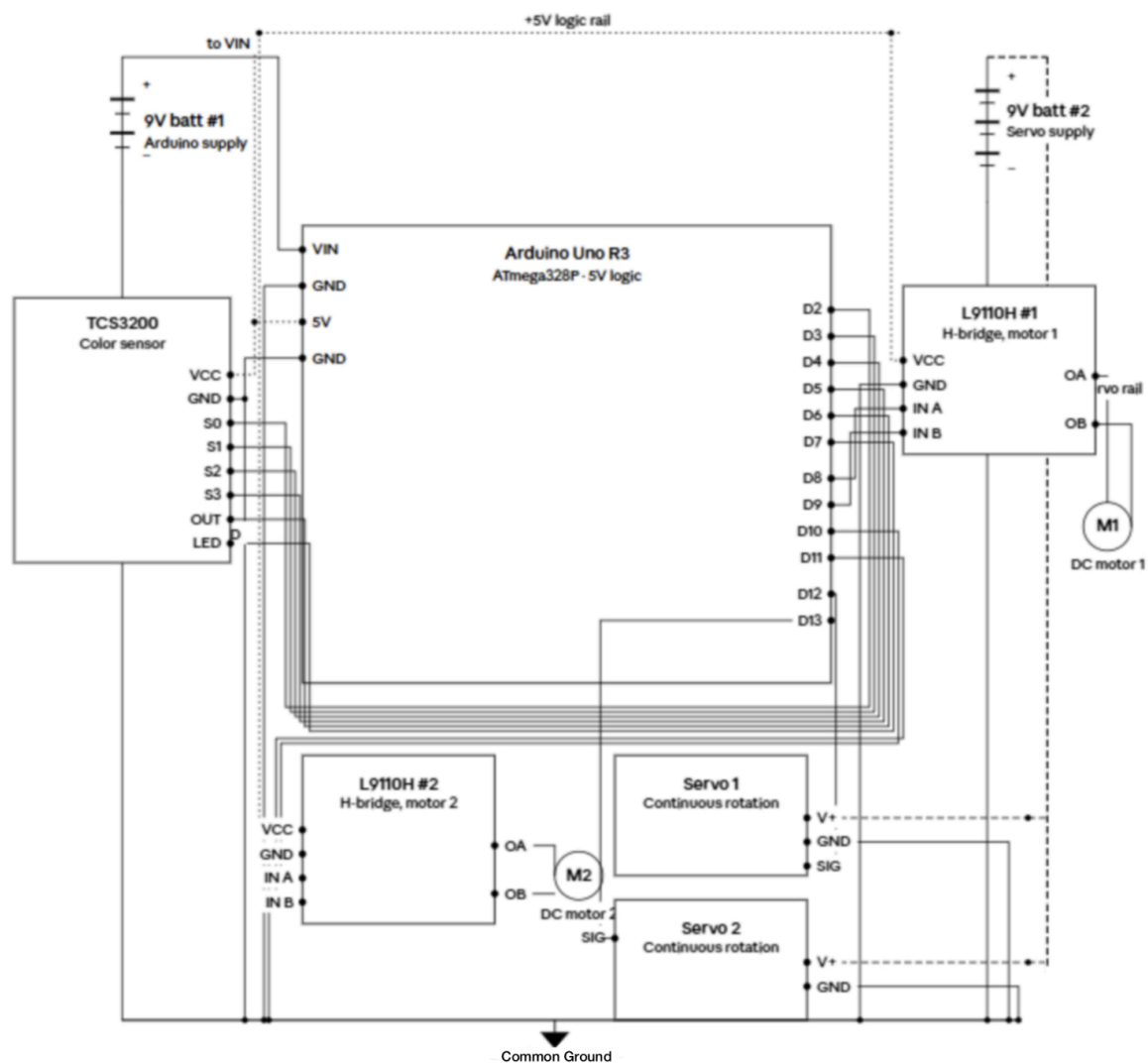


Diagram 1: Wiring diagram showing the connection between components, pins, inputs and outputs

7. Appendix C

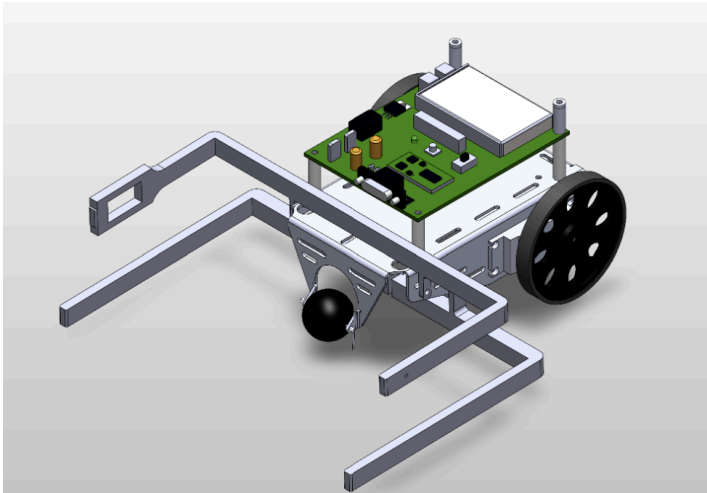


Figure 2: Full Assembly of the Manufactured Guards Integrated with Boe-Bot Chassis

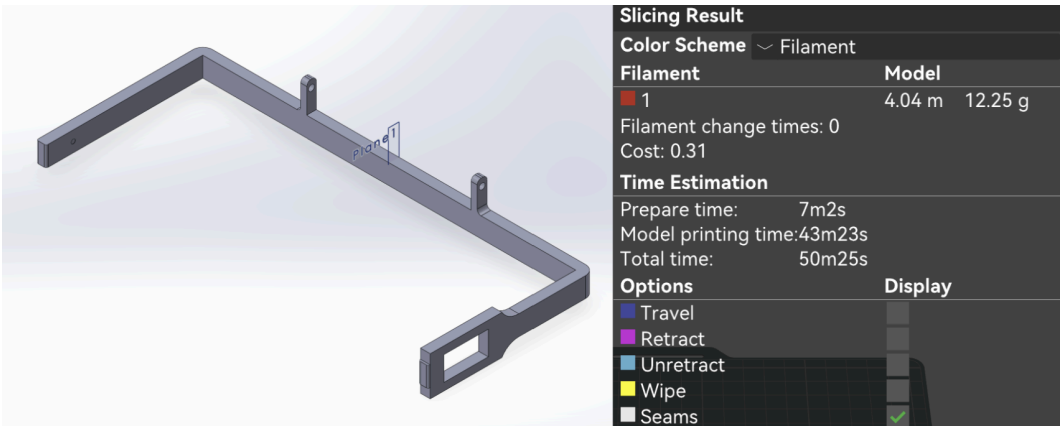


Figure 3: “U-Shaped Servo Holder” CAD with sliced mass estimate used in BOM

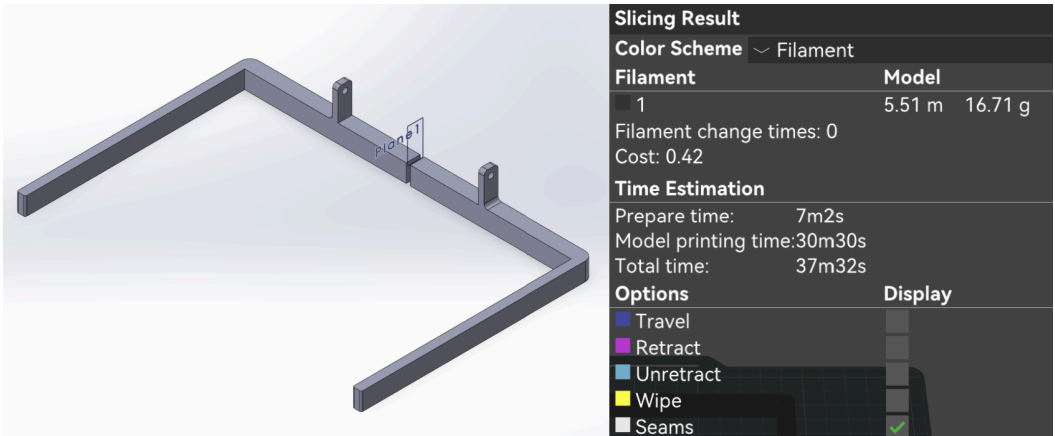


Figure 4: “Final U-Shaped Block Guard” CAD with sliced mass estimate used in BOM

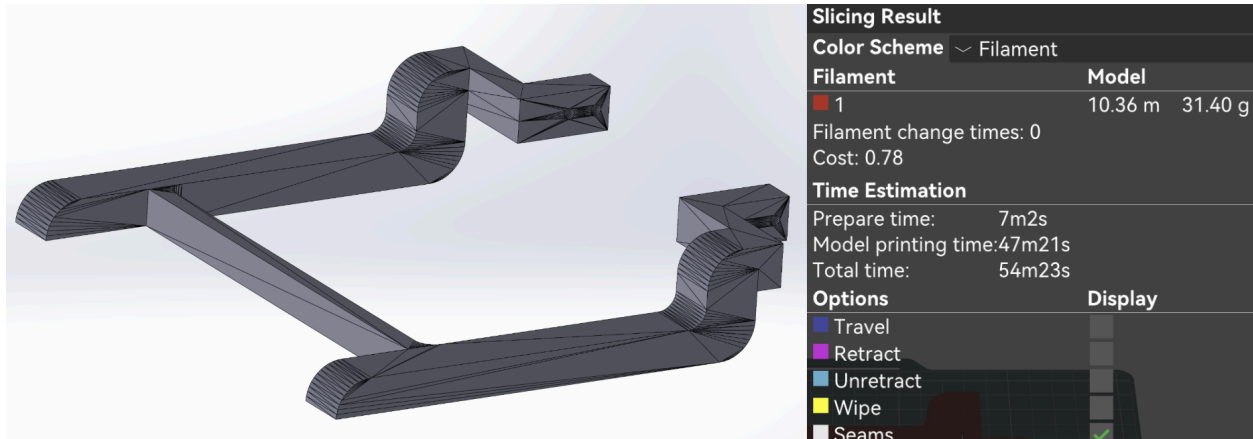
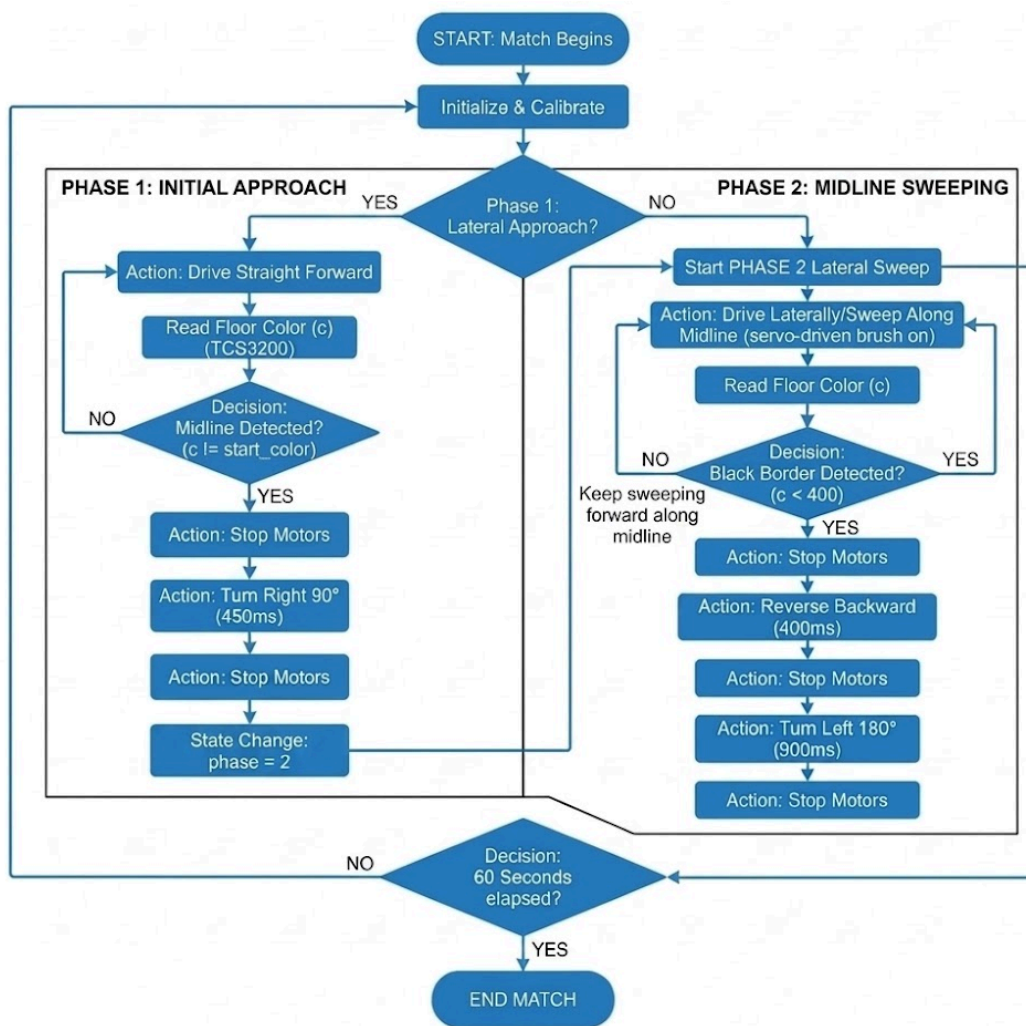


Figure 5: "Prototype Block Guard" CAD with sliced mass estimate used in BOM

8. Appendix D



9. Appendix E

```
#include <Servo.h> // Load the servo library.
#define S0 2 //Assign pin 2, 3, 4, 5 to S0, 1, 2, 3.
#define S1 3
#define S2 4
#define S3 5
#define COLOR_OUT 6 //Assign pin 6 to read sensor output
#define COLOR_LED 7 //Assign pin 7 to the sensor LED
#define SERVO_PIN 12 //Assign pin 12 to the servo
Servo sweepServo; //Make a servo object.
void drive_forward() { PORTB = (PORTB & 0xF0) | 0b00000101; } //Move robot forward
void drive_backward() { PORTB = (PORTB & 0xF0) | 0b00001010; } //Move robot
backward
void turn_left() { PORTB = (PORTB & 0xF0) | 0b00000110; } //Spin robot to the left
void turn_right() { PORTB = (PORTB & 0xF0) | 0b00001001; } //Spin robot to the right
void stop_motors() { PORTB = PORTB & 0xF0; } //Turn off the wheel motors
unsigned int count_pulses(unsigned int ms) { //Count sensor signals over a time window
    unsigned int count = 0; //Start pulse count at zero
    bool last = digitalRead(COLOR_OUT); //Get the starting pin state
    unsigned long end = millis() + ms; //Calculate when to stop counting
    while (millis() < end) { //Loop until the time is up
        bool now = digitalRead(COLOR_OUT); //Read the current pin state
        if (now != last) { count++; last = now; } //Add to count if the state changed
    }
    return count / 2; //Return full pulse cycles
}
char read_color() { //Check what color is under the robot
    digitalWrite(S2, HIGH); //Set sensor filter pins
    digitalWrite(S3, LOW); //Set sensor filter pins
    unsigned int p = count_pulses(50); //Get pulse count for 50 milliseconds
    Serial.print("Pulses: "); Serial.println(p); //Print the raw sensor number
    if (p < 400) return 'K'; //Return K if it sees black (we tested the color sensor and
found out that black values around 400 and yellows values around 660)
    if (p <= 660) return 'Y'; //Return Y if it sees yellow
    return 'B'; //Return B if it sees blue
}
char start_color; //Store the color we start on
int phase = 1; //Start in the first movement phase
void setup() { //Run setup code once
    DDRB |= 0x0F; //Set the motor pins as outputs
    stop_motors(); //Make sure motors are off at the start
    pinMode(S0, OUTPUT); pinMode(S1, OUTPUT); //Set up sensor control pins
    pinMode(S2, OUTPUT); pinMode(S3, OUTPUT); //Set up sensor control pins
    pinMode(COLOR_OUT, INPUT); //Set up the pin to read sensor data
    pinMode(COLOR_LED, OUTPUT); //Set up the pin to turn on sensor light
    sweepServo.attach(SERVO_PIN); //Tell the board where the servo is plugged in
```

```

sweepServo.writeMicroseconds(1700); //Start spinning the brush mechanism
digitalWrite(S0, HIGH); digitalWrite(S1, LOW); //Set sensor frequency scaling
digitalWrite(S2, HIGH); digitalWrite(S3, LOW); //Clear color filter settings
digitalWrite(COLOR_LED, HIGH); //Turn the sensor light on
Serial.begin(9600); //Start serial communication for debugging
delay(2000); //Wait two seconds before doing anything
start_color = read_color(); //Check the starting floor color
Serial.print("Start: "); //Print a label to the monitor
Serial.println(start_color); //Print the starting color to the monitor
delay(500); //Wait half a second
drive_forward(); //Start driving straight
}
void loop() { //Run this code over and over
  char c = read_color(); //Get the current floor color
  if (phase == 1) { //Check if we are still looking for the midline
    bool crossed = (start_color == 'B' && c == 'Y') || //Check if we went from blue to
yellow
    (start_color == 'Y' && c == 'B'); //Check if we went from yellow to blue
    if (crossed) { //Run this if the color changed
      stop_motors(); delay(100); //Stop briefly
      turn_right(); delay(450); //Turn 90 degrees to follow the line
      stop_motors(); delay(100); //Stop briefly again
      phase = 2; //Switch to the sweeping phase
    }
    drive_forward(); //Keep driving if we have not crossed yet
    return; //Skip the rest of the loop for now
  }
  if (c == 'K') { //Run this if we hit the black border
    stop_motors(); delay(100); //Stop moving
    drive_backward(); delay(400); //Back away from the edge
    stop_motors(); delay(100); //Stop briefly
    turn_left(); delay(900); //Turn 180 degrees to sweep the other way
    stop_motors(); delay(100); //Stop briefly
  }
  drive_forward(); //Keep moving forward along the line
}

```